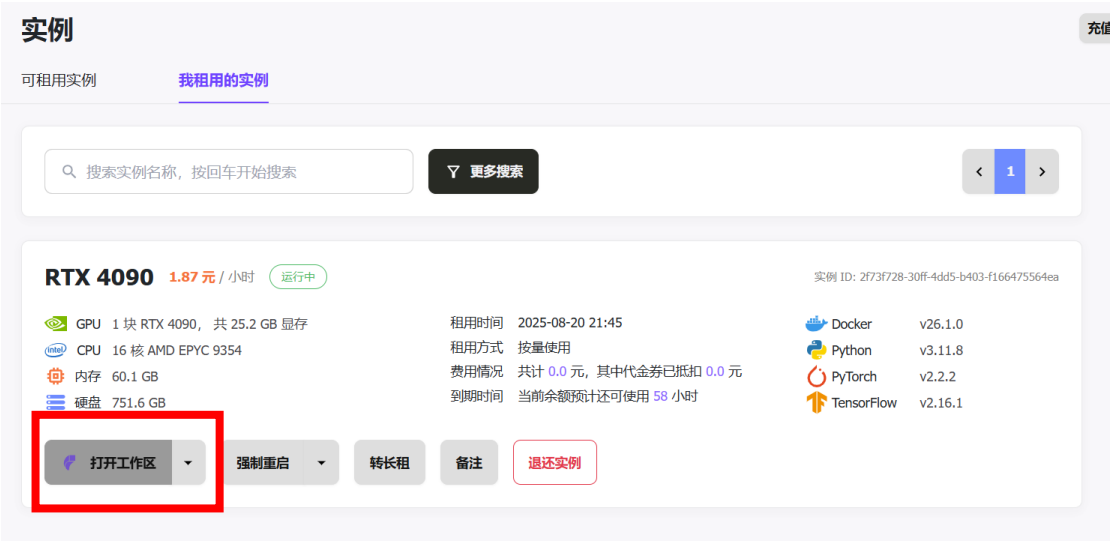
实验九. 基于 yolov5 的红绿灯识别

第二部分 模型训练（云端）

3. 训练模型（云端进行）

1）准备工作

登陆 https://featurize.cn/后，打开工作区，



点击下方“终端”



逐条输入：

git clone https://github.com/ultralytics/yolov5

cd yolov5/

git checkout v7.0

创建并设置虚拟环境，运行

conda create -n yolov5 python==3.11 -y

conda activate yolov5

```
(base) ➜  ~ git clone https://github.com/ultralytics/yolov5
Cloning into 'yolov5'...
remote: Enumerating objects: 17544, done.
remote: Counting objects: 100% (42/42), done.
remote: Compressing objects: 100% (39/39), done.
remote: Total 17544 (delta 25), reused 3 (delta 3), pack-reused 17502 (from 2)
Receiving objects: 100% (17544/17544), 16.66 MiB | 15.43 MiB/s, done.
Resolving deltas: 100% (12022/12022), done.
(base) ➜  ~ cd yolov5
(base) ➜  yolov5 git checkout v7.0
Note: switching to 'v7.0'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 915bbf29 YOLOv5 v7.0 release updates (#10245)
(base) ➜  yolov5 git branch
(base) ➜  yolov5
```

安装依赖，运行

pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple

conda install pytorch torchvision torchaudio pytorch-cuda -c pytorch -c nvidia -y

下载官方的预训练权重

wget https://github.com/ultralytics/yolov5/releases/download/v7.0/yolov5n.pt

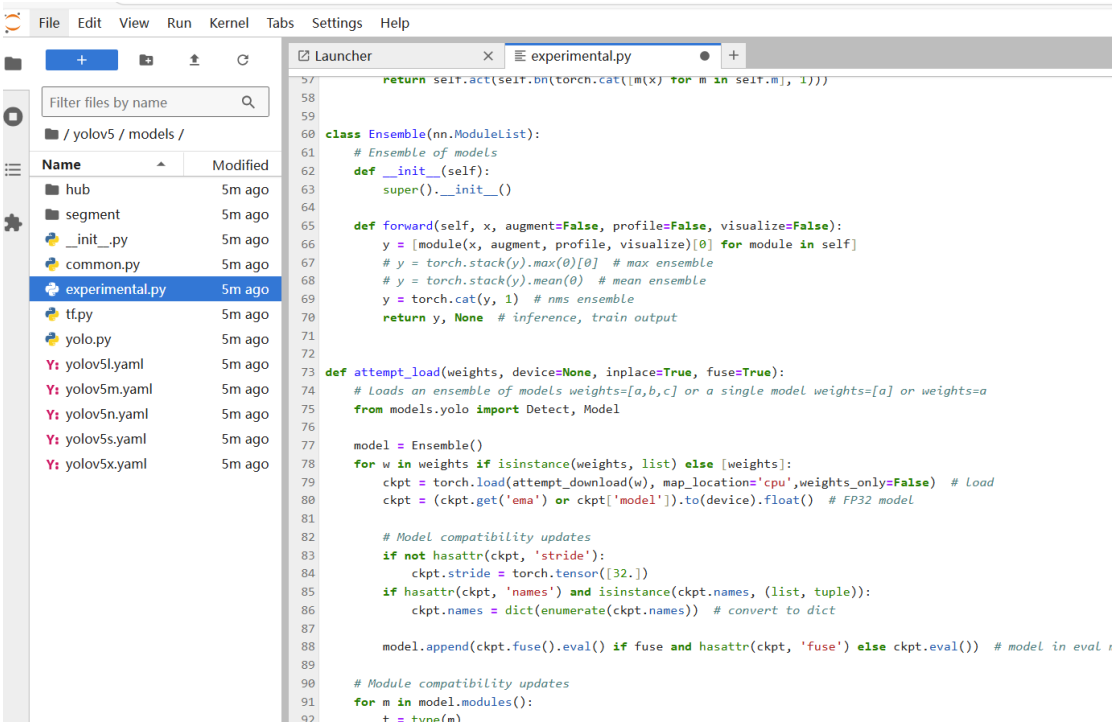
点击 JupyterLab



打开.\yolov5\ models\experimental.py，在第 79 行，修改为

```
77      model = Ensemble()
78      for w in weights if isinstance(weights, list) else [weights]:
79          ckpt = torch.load(attempt_download(w), map_location='cpu', weights_only=False)  # load
80          ckpt = (ckpt.get('ema') or ckpt['model']).to(device).float()  # FP32 model
81
```

Ctr+s 存盘后，退出



开始验证安装是否正确，在终端界面，运行

python detect.py --source ./data/images/ --weights yolov5n.pt --conf 0.4

```
(yolov5) C:\Users\hchuang\yolov5>python detect.py --source ./data/images/ --weights yolov5n.pt --conf 0.4
detect: weights=['yolov5n.pt'], source=./data/images/, data=data\coco128.yaml, imgsz=[640, 640], conf_thres=0.4, iou_thr
es=0.45, max_det=1000, device=, view_img=False, save_txt=False, save_conf=False, save_crop=False, nosave=False, classes=
None, agnostic_nms=False, augment=False, visualize=False, update=False, project=runs\detect, name=exp, exist_ok=False, l
ine_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False, vid_stride=1
YOLOv5  v7.0-0-g915bbf29 Python-3.11.0 torch-2.5.1 CUDA:0 (NVIDIA GeForce GTX 1070 with Max-Q Design, 8192MiB)

C:\Users\hchuang\yolov5\models\experimental.py:79: FutureWarning: You are using `torch.load` with `weights_only=False` (
the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickl
e data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md
#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`
. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be
loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We r
ecommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Pl
ease open an issue on GitHub for any issues related to this experimental feature.
  ckpt = torch.load(attempt_download(w), map_location='cpu')  # load
Fusing layers...
YOLOv5n summary: 213 layers, 1867405 parameters, 0 gradients, 4.5 GFLOPs
image 1/2 C:\Users\hchuang\yolov5\data\images\bus.jpg: 640x480 4 persons, 1 bus, 42.9ms
image 2/2 C:\Users\hchuang\yolov5\data\images\zidane.jpg: 384x640 2 persons, 39.6ms
Speed: 1.0ms pre-process, 41.3ms inference, 92.3ms NMS per image at shape (1, 3, 640, 640)
Results saved to runs\detect\exp
```

执行完成后就可以在./runs/detect/exp 看到检测到的图像了。





2）模型训练

上传 traffice_sign.zip 到 yolov5/data 目录下

在终端下，运行

cd data

unzip traffice_sign.zip

```
(yolov5) ➜ yolov5 cd data
(yolov5) ➜ data unzip traffice_sign.zip
```

然后需要准备自己的配置，在首先是模型的配置，在 JupyterLab 界面下，将./models/yolov5n.yaml 复制为 myyolov5n.yaml，修改 nc 值，设置为标注的类别的数量，本实验红绿 3 个交通灯故设置为 3。

```
# Parameters
nc: 3  # number of classes
depth_multiple: 0.33  # model depth multiple
width_multiple: 0.25  # layer channel multiple
```

在 data 目录下，新建文件 mydata.yaml（建议在本地计算机上，建好了上传）

```
train: ./data/traffice_sign/train/images
val: ./data/traffice_sign/val/images
test: ./data/traffice_sign/test/images


# number of classes
```

nc: 3

class names

names: ['green-light', 'blue-light','red-light']

修改 yolov5/train.py 文件

```
('--weights', type=str, default=ROOT / 'yolov5n.pt', help='initial weights path')
('--cfg', type=str, default='myyolov5n.yaml', help='model.yaml path')
('--data', type=str, default=ROOT / 'data/mydata.yaml', help='dataset.yaml path')
('--hyp', type=str, default=ROOT / 'data/hyps/hyp.scratch-low.yaml', help='hyperpara
('--epochs', type=int, default=300, help='total training epochs')
('--batch-size', type=int, default=16, help='total batch size for all GPUs, -1 for a
('--imgsz', '--img', '--img-size', type=int, default=672, help='train, val image siz
('--rect', action='store_true', help='rectangular training')
('--resume', nargs='?', const=True, default=False, help='resume most recent training
('--nosave', action='store_true', help='only save final checkpoint')
('--noval', action='store_true', help='only validate final epoch')
('--noautoanchor', action='store_true', help='disable AutoAnchor')
('--noplots', action='store_true', help='save no plot files')
('--evolve', type=int, nargs='?', const=300, help='evolve hyperparameters for x gene
('--bucket', type=str, default='', help='gsutil bucket')
('--cache', type=str, nargs='?', const='ram', help='image --cache ram/disk')
('--image-weights', action='store_true', help='use weighted image selection for trai
('--device', default='0', help='cuda device, i.e. 0 or 0,1,2,3 or cpu')
```

其中

--weights 为刚刚下载的 yolov5n.pt

--cfg 和--data 为刚才的 myyolov5nyaml 和 mydata.yaml,

--epochs 和--batch-size 根据自己的电脑配置来选,

--imgsz，改为 672

--device 选择 0 表示用 gpu 训练。没有选 cpu。

修改 137 行

```
122             weights = attempt_download(weights)  # download if not found locally
123         ckpt = torch.load(weights, map_location='cpu',weights_only=False)  # load checkp
```

开始训练

在 yolov5 目录下，运行

python train.py

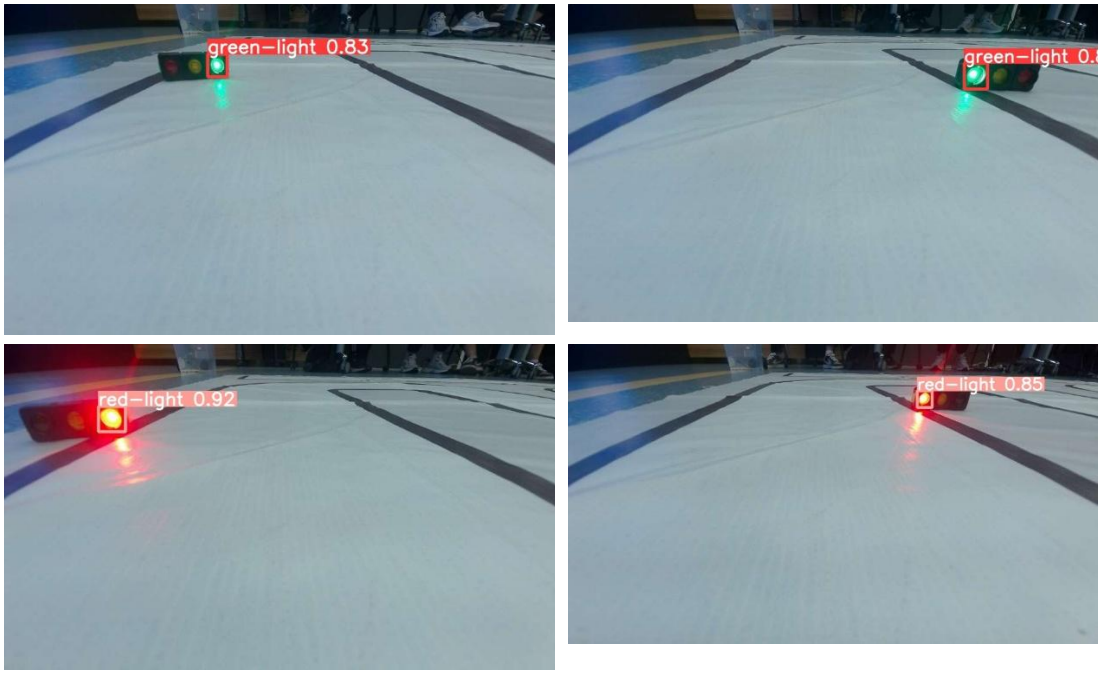
```
(yolov5)
(yolov5) ➜ data cd ..
(yolov5) ➜ yolov5 python train.py
```

训练完成后，会在.\runs\train 文件夹下有 exp 文件夹，里面用来放训练的结果，我们需要用到的即为 expX\weights\best.pt（X 代表最大的数字）。拷贝 best.pt 到 yolov5 目录下。

先验证一下模型的准确性，运行

python detect.py --source ./data/traffice_sign/test/images --weights ./best.pt --conf 0.4

在\runs\detect\expX 中观察输出结果。



接下来进行模型转换，首先先将原始的 pt 模型转 onnx 模型

修改./models/yolo.py 文件, Detect 类, forward 方法，约 56 行. 注：建议您保留好原本的 forward 方法,例如改一个其他的名字 forward_，方便在训练的时候换回来。

```
def forward(self, x):
    return [self.m[i](x[i]).permute(0,2,3,1).contiguous() for i in
range(self.nl)]
```

```
56      def forward(self, x):
57          return [self.m[i](x[i]).permute(0,2,3,1).contiguous() for i in range(self.nl
58
59      def forward_(self, x):
60          z = []  # inference output
61          for i in range(self.nl):
```

打开 yolov5/export.py 文件，作如下修改. 建议您在修改前先复制一份留存.

- 约 150 行，作如下修改

```
torch.onnx.export(
    model.cpu() if dynamic else model,  # --dynamic only compatible with cpu
    im.cpu() if dynamic else im,
    f,
    verbose=False,
    opset_version=opset,
    do_constant_folding=True,
    input_names=['images'],
    output_names=['small', 'medium', 'big'],
    dynamic_axes=dynamic or None)
```

- 约 552 行，注释掉以下两行

```
# shape = tuple((y[0] if isinstance(y, tuple) else y).shape)  # model
output shape
# LOGGER.info(f"\n{colorstr('PyTorch:')} starting from {file} with output
shape {shape} ({file_size(file):.1f} MB)")
```

```
552     #shape = tuple((y[0] if isinstance(y, tuple) else y).shape)  # m
553     metadata = {'stride': int(max(model.stride)), 'names': model.nam
554     #LOGGER.info(f"\n{colorstr('PyTorch:')} starting from {file} wit
```

- 约 612 行，作如下修改

```
parser.add_argument('--weights', nargs='+', type=str, default=ROOT /
'best.pt', help='model.pt path(s)')
parser.add_argument('--imgsz', '--img', '--img-size', nargs='+', type=int,
default=[672, 672], help='image (h, w)')
parser.add_argument('--simplify', default=True, action='store_true',
help='ONNX: simplify model')
parser.add_argument('--opset', type=int, default=11, help='ONNX: opset
version')
parser.add_argument('--include',
                    nargs='+',
                    default=['onnx'],
                    help='torchscript, onnx, openvino, engine, coreml,
saved_model, pb, tflite, edgetpu, tfjs')
```

运行

python export.py

yolov5 文件夹下会产生 best.onnx 模型。